

AI Vision Analysis Summary

Infrastructure Inspection Agent · NVIDIA NIM + Meta Llama 3.2 Vision integration

1. What replaced what

	Before	Now
Engine	Local Python server (AI-agent/ai_server.py) running SAM 2.1 (Segment Anything) + Grounding DINO	One hosted call to meta/llama-3.2-11b-vision-instruct on NVIDIA NIM (build.nvidia.com)
Requires	GPU, model weights (~800 MB), a dedicated long-lived process	Nothing — pure Node fetch, one API key
Deployable on	GPU VM only	Render / Vercel serverless

2. Pipeline — image in, structured inspection out

2.1 Request path

Officer opens the AI Vision tab for a complaint → frontend calls `POST /api/ai-agent/vision` with `{ imageUrl, title, category }` → `aiAgentController.visionInspect` → `nvidiaVisionEngine.detectDefectsNvidia()`.

2.2 Image handling

- **http(s) URL** (hosted photo) → passed straight to NIM as `image_url.url`; NVIDIA fetches it server-side.
- **base64 data: URI** (citizen phone upload) → passed inline if ≤ 180 KB.
- Neither / oversized → deterministic fallback (see §4).

2.3 The prompt — one vision-LLM call

System role: “infrastructure inspection vision model for a municipal corporation... respond with ONLY a compact JSON object.”

User message = the image + a strict JSON schema the model must fill, framed as an 8-stage municipal

inspection. Returned in one shot:

Field	Meaning
<code>infrastructureType</code> + <code>infrastructureConfidence</code>	ROAD / BRIDGE / BUILDING / DRAINAGE / WATER_PIPELINE / ELECTRICAL / STREET_LIGHT / OTHER
<code>surfaceRegion</code>	bounding box of the asset surface (image %, top-left origin)
<code>defects[]</code> (1–5, most severe first)	<code>defectType</code> , <code>severity</code> , <code>confidence</code> , <code>dimensions</code> , <code>lengthMeters</code> , <code>widthMeters</code> , <code>standard</code> (IRC/CPHEEO/CEA/SWM), <code>bbox</code> , <code>isCrack</code> , <code>hasWater</code>
<code>surroundings</code>	<code>cracksDetected</code> , <code>waterOrMoisture</code> , <code>deteriorationRating</code> , <code>inspectionZoneRadiusMeters</code> , <code>zoneCenter</code>
<code>thermal</code>	AI- inferred anomaly split <code>highAnomalyPct</code> / <code>moderatePct</code> / <code>nominalPct</code> , <code>riskLevel</code> , <code>hotspots[]</code> — inferred from RGB, no real thermal camera
<code>keyFindings[]</code> , <code>recommendations[]</code>	bullet lists
<code>summary</code>	2–3 sentence executive engineering assessment (per-image)
<code>recommendedAction</code>	single most important corrective action
<code>overallSeverity</code> , <code>riskLevel</code> , <code>overallConfidence</code>	roll-ups

Call params: `temperature 0.2`, `top_p 1`, `max_tokens 1024`, `stream false` — low temperature keeps the JSON well-formed and the assessment grounded, not creative.

2.4 Response parsing & hardening

- Extract the JSON object from the reply — tolerates stray text / code fences (slices first { to last }).
- Every field clamped / normalized:
 - `clampPct` → all coordinates forced into 0–100
 - `normalizeSeverity` → free text → CRITICAL / HIGH / MEDIUM / LOW
 - `normalizeInfraType` → free text → one of 8 buckets
 - thermal percentages re-normalized to sum 100
 - missing values derived — zone center = defect centroid, hotspots = defect centers, deterioration = f(severity)
- Output = the full 8-stage payload the frontend renders.

3. Reliability layer

Mechanism	Behavior
Key pool + rotation	NVIDIA_API_KEYS (comma-separated) used round-robin; a request hitting 401 / 403 / 408 / 429 / 5xx rotates to the next key and retries.
Per-request timeout	AbortController at NVIDIA_TIMEOUT_MS (default 9000 ms) so a stalled key can't freeze the Vercel function (Hobby kills at 10 s). The 11B model was chosen over 90B to finish inside that window.
Deterministic fallback	No key / no usable image / all keys fail / unparseable JSON → the engine still returns the identical full shape so the tab always renders. Fallback summary, keyFindings, recommendations, recommendedAction are derived from category + defect type + severity + moisture flag (not one fixed string); the failure reason is tagged in engine.

The engine field tells you which path ran:

- nvidia:meta/llama-3.2-11b-vision-instruct → live model, per-image summary
- fallback (<reason>) → deterministic, category-derived summary
- nvidia-unparsed → model replied but not valid JSON

4. How the analysis surfaces in the UI

AIVisionInspectorCanvas maps each payload section to a stage. All overlays are CSS/SVG in image-% space — no server-rendered images.

Stage	Source field	Rendered as
1 Image	—	source photo
2 Infrastructure	infrastructure	blue surface-region box + type / confidence label
3 Defects	visionDefects[].box	red / yellow bounding boxes with type + confidence tags
4 Segmenting	same	pulsing dashed contour outlines
5 Surroundings	surroundings	dashed yellow inspection-zone ellipse + water marker
6	measurements	green crosshairs + L/W metre labels

Measurements

7		
Radiothermal	thermal	CSS gradient thermal tint + hotspot radials + anomaly %
8 Master	everything	all overlays stacked, 8 layer toggles, legend, executive synthesis panel (stats + summary + keyFindings + recommendations + recommendedAction + riskLevel)

5. Configuration

```
NVIDIA_API_KEYS      = nvapi-...,nvapi-...      (pool, round-robin + failover)
NVIDIA_VISION_MODEL  = meta/llama-3.2-11b-vision-instruct
NVIDIA_TIMEOUT_MS    = 9000                      (< Vercel Hobby 10s function cap)
NVIDIA_BASE_URL      = https://integrate.api.nvidia.com/v1  (default)
```

Endpoint: POST /api/ai-agent/vision → { success, data: <8-stage payload> }

Key files: backend/services/nvidiaVisionEngine.js, backend/controllers/aiAgentController.js (visionInspect), backend/routes/aiAgent.js (POST /vision), frontend/src/components/agent/AIVisionInspectorCanvas.jsx.